

Image Storage in Turso Database

Overview

Images are now stored directly in **Turso database** as BLOB data and served through **Rust edge functions**. This provides:

- No separate file storage needed
 - Images backed up with your database
 - Edge-deployed image serving
 - Global CDN distribution (via Turso's edge network)
 - Consistent backup/restore with database
-

Database Schema

Images Table

```
CREATE TABLE images (  
  id TEXT PRIMARY KEY,           -- UUID  
  filename TEXT NOT NULL,        -- Original filename  
  content_type TEXT NOT NULL,     -- MIME type (image/jpeg, etc.)  
  data BLOB NOT NULL,            -- Binary image data  
  size INTEGER NOT NULL,         -- File size in bytes  
  uploaded_by TEXT,              -- User ID (optional)  
  created_at INTEGER NOT NULL,   -- Unix timestamp  
  FOREIGN KEY (uploaded_by) REFERENCES users(id)  
);
```

How It Works

Upload Flow

1. User selects image in admin panel
2. Frontend converts to base64
3. API receives base64 data
4. Rust validates file type & size
5. Decoded to binary and stored in Turso
6. Returns image ID and URL
7. Frontend displays preview

Retrieval Flow

1. Browser requests /api/images/{id}
2. Rust edge function queries Turso
3. Image BLOB retrieved from database

4. Served with correct headers:
- Content-Type (image/jpeg, etc.)
 - Cache-Control (1 year cache)
 - Content-Length
-

API Endpoints

Upload Image

POST /api/upload/image

Request:

```
{
  "filename": "photo.jpg",
  "content_type": "image/jpeg",
  "data": "base64_encoded_image_data"
}
```

Response:

```
{
  "success": true,
  "data": {
    "id": "uuid-here",
    "url": "http://localhost:3000/api/images/uuid-here",
    "filename": "photo.jpg"
  }
}
```

Get Image

GET /api/images/{id}

Response: - Binary image data - Headers: - Content-Type: image/jpeg -
Cache-Control: public, max-age=31536000, immutable - Content-Length:
12345

Delete Image

DELETE /api/images/{id}

Response:

```
{
  "success": true
}
```

Frontend Usage

ImageUpload Component

```
import ImageUpload from '@components/ImageUpload'
```

```
<ImageUpload  
  onImageUploaded={(url) => setImageUrl(url)}  
  currentImage={imageUrl}  
>
```

Features: - Drag & drop (coming soon) - File validation - Preview - Progress indicator - Remove image - Error handling

Manual Upload

```
const uploadImage = async (file: File) => {  
  // Convert to base64  
  const reader = new FileReader()  
  reader.onload = async (e) => {  
    const base64 = e.target?.result?.toString().split(',')[1]  
  
    const response = await fetch('/api/upload/image', {  
      method: 'POST',  
      headers: { 'Content-Type': 'application/json' },  
      body: JSON.stringify({  
        filename: file.name,  
        content_type: file.type,  
        data: base64  
      })  
    })  
  
    const result = await response.json()  
    console.log('Image URL:', result.data.url)  
  }  
  reader.readAsDataURL(file)  
}
```

Validation & Limits

File Type Validation

Allowed: - image/jpeg - image/png - image/gif - image/webp

Not Allowed: - SVG (security risk) - Other file types

Size Limits

- **Maximum:** 5MB per image
- **Recommended:** Under 1MB for best performance
- **Turso Limit:** Row size up to 1GB (plenty of room)

Validation Flow

```
// 1. Check content type
if !content_type.starts_with("image/") {
  return Error("Only images allowed")
}

// 2. Decode base64
let data = decode(base64)?;

// 3. Check size
if data.len() > 5 * 1024 * 1024 {
  return Error("Max 5MB")
}

// 4. Store in database
image_repo.create(filename, content_type, data)?;
```

Performance

Caching

Browser Caching:

Cache-Control: public, max-age=31536000, immutable

This tells browsers to cache for 1 year since image IDs are immutable.

Edge Distribution

- Images stored in Turso (LibSQL)
- Turso has edge locations worldwide
- Low latency globally
- No separate CDN needed

Optimization Tips

1. **Compress before upload** - Use tools like ImageOptim
2. **Use appropriate formats:**
 - Photos: JPEG

- Graphics: PNG
 - Animations: GIF or WebP
3. **Resize to needed dimensions** - Don't upload 4K for thumbnails
 4. **Use responsive images** in HTML:

```
<img
  srcset="image-small.jpg 400w, image-large.jpg 800w"
  sizes="(max-width: 600px) 400px, 800px"
/>
```

Storage Comparison

Turso Database (Current)

Pros: - All-in-one solution - Automatic backups - Edge distribution - No separate service - Consistent with data

Cons: - 5MB per image limit (by choice) - No built-in image processing - Takes database space

Alternative: Vercel Blob

Pros: - 100GB free tier - Automatic optimization - Built for large files

Cons: - Separate service - More complexity - Separate backups

Alternative: Cloudinary

Pros: - Image transformations - 25GB free tier - Automatic optimization

Cons: - Separate service - External dependency - API complexity

Migration Guide

From File System to Turso

If you have existing images in `/uploads/images`:

```
# Run migration script (create this)
cargo run --bin migrate-images
```

Migration Script:

```
// Read all files from uploads/images
// For each file:
//   - Read as bytes
//   - Detect content type
```

```
// - Insert into database
// - Update post references
```

From External CDN to Turso

1. Download all images
 2. Upload via API
 3. Update post URLs
 4. Verify all images load
 5. Remove old CDN
-

Administration

List All Images

```
SELECT
  id,
  filename,
  size,
  uploaded_by,
  created_at
FROM images
ORDER BY created_at DESC;
```

Check Storage Usage

```
SELECT
  COUNT(*) as total_images,
  SUM(size) as total_bytes,
  SUM(size) / 1024 / 1024 as total_mb
FROM images;
```

Find Large Images

```
SELECT
  filename,
  size / 1024 / 1024 as size_mb
FROM images
WHERE size > 1024 * 1024 -- Larger than 1MB
ORDER BY size DESC
LIMIT 10;
```

Delete Orphaned Images

```
-- Find images not referenced by any post
SELECT i.id, i.filename
```

```

FROM images i
WHERE NOT EXISTS (
    SELECT 1 FROM posts p
    WHERE p.featured_image LIKE '%' || i.id || '%'
);

```

Troubleshooting

Image Won't Upload

Check: 1. File size < 5MB 2. Valid image format 3. Base64 encoding correct
4. Database connection working

Debug:

```

# Test upload
curl -X POST http://localhost:3000/api/upload/image \
  -H "Content-Type: application/json" \
  -d '{"filename":"test.jpg","content_type":"image/jpeg","data":"..."}'

```

Image Won't Display

Check: 1. Image ID is correct 2. Database has the image 3. Content-Type header set 4. CORS allows the domain

Debug:

```

# Get image info
sqlite3 blog.db "SELECT id, filename, content_type, size FROM images WHERE id='...'"

# Test retrieval
curl -I http://localhost:3000/api/images/uuid-here

```

Images Load Slowly

Solutions: 1. Compress images before upload 2. Use browser caching (already enabled) 3. Consider image proxy/CDN 4. Implement lazy loading

Security

Current Protection

- File type validation
- Size limits
- Base64 decoding errors handled
- SQL injection prevented (parameterized queries)

Recommendations

1. **Add authentication** - Only logged-in users upload
2. **Scan for malware** - Integrate virus scanning
3. **Rate limiting** - Prevent upload spam
4. **Content validation** - Verify actual image data
5. **Access control** - Who can delete images?

Authentication Example

```
// In upload handler
let user_id = extract_user_from_jwt(request)?;
image_repo.create(filename, content_type, data, Some(&user_id))
```

Turso Limits

Free Tier

- **Databases:** 500
- **Row reads:** 1 billion/month
- **Row writes:** 25 million/month
- **Storage:** 9 GB total

Image Storage Estimate

5MB image = 1 database write

1000 images = 5GB storage

With free tier: ~**1800 images** (9GB / 5MB)

For more images: Upgrade or compress more aggressively.

Best Practices

1. Optimize Before Upload

```
// Use client-side compression
import imageCompression from 'browser-image-compression'

const compressedFile = await imageCompression(file, {
  maxSizeMB: 1,
  maxWidthOrHeight: 1920
})
```


2. Generate Thumbnails

Store multiple sizes: - Thumbnail: 150x150 - Medium: 800x600 - Large: 1920x1080

3. Lazy Load Images

```

```

4. Use Descriptive Filenames

```
// Good
const filename = `blog-post-hero-${Date.now()}.jpg`

// Bad
const filename = `IMG_1234.jpg`
```

5. Clean Up Unused Images

Run periodic cleanup:

```
DELETE FROM images
WHERE created_at < unixepoch('now', '-30 days')
AND NOT EXISTS (
  SELECT 1 FROM posts WHERE featured_image LIKE '%' || images.id || '%'
);
```

Production Deployment

Vercel Configuration

vercel.json:

```
{
  "functions": {
    "api/images/*": {
      "maxDuration": 10
    }
  }
}
```

Environment Variables

```
# Backend .env
TURSO_DATABASE_URL=libsql://your-db.turso.io
TURSO_AUTH_TOKEN=your-token
API_BASE_URL=https://api.yourdomain.com
```

```
# Frontend .env
NEXT_PUBLIC_API_URL=https://api.yourdomain.com
```

Monitoring

Track: - Upload success rate - Average image size - Storage usage - Retrieval latency

Future Enhancements

Planned: - ☐ Image thumbnails generation - ☐ Multiple image formats (WebP conversion) - ☐ Image optimization pipeline - ☐ Drag & drop upload - ☐ Batch upload - ☐ Image gallery in admin - ☐ Usage analytics - ☐ Automatic cleanup of orphaned images

Questions? Check the main documentation or open an issue!

Performance Issues? Consider external CDN for large-scale deployments.

Need More Storage? Upgrade Turso plan or implement hybrid approach (Turso for metadata, S3 for BLOBs).